Kazakh-British Technical University

Data collection and preparation

STUDENTS' INDEPENDENT STUDY (SIS) №2

Pink Floyd Physical Releases - Kafka and Pandera Pipeline

Full Name: Nursultan Zhantuar

ID: 23B030356

12.11.2026

Almaty

Setup - Environment and Configuration

In this project I built one connected, finite data pipeline around the **MusicBrainz API**: *MusicBrainz API -> raw JSONL -> Kafka producer -> Apache Kafka broker/topic -> Kafka consumer -> Pandera -> pandas/NumPy -> SQLite3 -> EDA figures*. The code is written in **Python 3.11/3.12**, the collection uses *requests*, the messages go through a real **Apache Kafka 3.9.1** broker started with **Docker Compose**, the validation is done with **Pandera**, the cleaning with **pandas** and **NumPy**, the storage with **SQLite3**, and the figures with **Matplotlib** and **Seaborn**.

The repository has a clear structure: *main.py* is the entry point, the modules of the pipeline are in *src/* (api_client, producer, consumer, validator, cleaner, loader, analysis), the raw and processed files are in *data/*, the figures in *figures/*, the SQL checks in *sql/verification_queries.sql*, and the tests in *tests/*. I created a virtual environment, installed the dependencies from *requirements.txt* and started the broker with *docker compose up -d*. The dependencies are not pinned because of version problems I had during development, which is a known limitation of the project.

Clone the repo and crate a virtual python environment

```
git clone https://github.com/n-n06/data-collection-sis2
cd pink-floyd-kafka-pandera
python -m venv .venv
```

Activate the virtual environment:

```
# Linux/macOS
source .venv/bin/activate

# Windows PowerShell
.venv\Scripts\Activate.ps1
```

Install dependencies:

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

```
PS C:\Users\nursultan.zhantuar> git clone https://github.com/n-n06/data-collection-sis2
Cloning into 'data-collection-sis2'...
remote: Enumerating objects: 77, done.
remote: Counting objects: 100% (77/77), done.
remote: Compressing objects: 100% (63/63), done.
remote: Total 77 (delta 23), reused 61 (delta 11), pack-reused 0 (from 0)
Receiving objects: 100% (77/77), 166.61 KiB | 541.00 KiB/s, done.
Resolving deltas: 100% (23/23), done.
PS C:\Users\nursultan.zhantuar> cd data-collection-sis2
PS C:\Users\nursultan.zhantuar\data-collection-sis2> python -m venv .venv
```

Task 1 - Project Topic, Data Source and Pipeline Design

The topic of the project is **physical Pink Floyd release editions**: CD, vinyl, cassette, MiniDisc, 8-track and DVD/Blu-ray. The goal is not only to analyze the releases, but mainly to build a reliable pipeline, in which every stage can be checked: the data must be collected from a public API, sent as messages through Kafka, validated, cleaned, loaded into a database without duplicates and finally analyzed.

Data source

The source is the public **MusicBrainz API** (https://musicbrainz.org). No API key is required. Digital Media and streaming releases are filtered out on purpose, and releases without a date are rejected, so the pipeline collects only dated physical releases. The target is **250 records**.

Pipeline stages

- **Collection** - the API client requests the releases and preserves every original API page and the normalized (but uncleaned) records.
- **Kafka producer** - sends every record as a JSON message to the topic *pink-floyd-physical-releases*.
- **Kafka consumer** - reads exactly the expected number of messages of the current run and stops.
- **Pandera validation** - checks every record against 14 rules and quarantines the invalid ones.
- **Cleaning and transformation** - pandas and NumPy prepare the valid rows.
- **SQLite loading** - the processed table is loaded idempotently and checked with SQL.
- **EDA** - four figures and statistics are generated.

The repository also contains a deterministic 250-record **snapshot** of the MusicBrainz metadata. The live collection (*--source api*) is the required mode, but the snapshot (*--source snapshot*) makes the reported results reproducible even if the live catalog changes, and it follows exactly the same Kafka path.

Task 2 - API Data Collection

Endpoint and parameters

The client in *src/api_client.py* sends GET requests to *https://musicbrainz.org/ws/2/release* with the following parameters:

- *artist* = 83d91898-7763-47d7-b03b-b92132375c47 (the MusicBrainz ID of Pink Floyd);
- *inc* = media+release-groups+labels+artist-credits (the nested data to include);
- *fmt* = json;

- *limit* = 100 and *offset* = 0, 100, 200, ... (pagination).

Reliability of the client

The client retries temporary HTTP failures with exponential backoff, checks unsuccessful responses, uses a 45-second timeout and waits about 1.1 seconds between requests to respect the MusicBrainz rate limit. It filters out Digital Media, rejects releases without a date, deduplicates release IDs and stops only when the requested target is reached. Every original API page is saved in *data/raw_api_pages/*, and the normalized but uncleaned records are saved in both *data/raw_data.jsonl* and *data/raw_data.csv*, so the raw data is never lost.

```
n_n@fedora:~/Projects/dcp-sis2/data-collection-sis2$ source venv/bin/activate
(venv) n_n@fedora:~/Projects/dcp-sis2/data-collection-sis2$ python main.py full --source api --target 250 --inject-invalid 3
Collected 250 dated physical Pink Floyd releases → /home/n_n/Projects/dcp-sis2/data-collection-sis2/data/raw_data.jsonl
Published 253 JSON messages to topic 'pink-floyd-physical-releases' for run_id=c4736ddc-a6f0-4ce9-a943-907ca622a384
Consumed 253/253 JSON messages from topic 'pink-floyd-physical-releases' for run_id=c4736ddc-a6f0-4ce9-a943-907ca622a384
/home/n_n/Projects/dcp-sis2/data-collection-sis2/src/validator.py:22: FutureWarning: Importing pandas-specific classes and functions from the
top-level pandera module will be **removed in a future version of pandera**.
If you're using pandera to validate pandas objects, we highly recommend updating
your import:

```
# old import
import pandera as pa

# new import
import pandera.pandas as pa
```

If you're using pandera to validate objects from other compatible libraries
like pyspark or polars, see the supported libraries section of the documentation
for more information on how to import pandera:

https://pandera.readthedocs.io/en/stable/supported_libraries.html

To disable this warning, set the environment variable:

```
export DISABLE_PANDERA_IMPORT_WARNING=True
```

  return DataFrameSchema(
Pandera validation: 250 valid, 3 invalid
Cleaning: 250 valid → 250 processed; removed 0 duplicates
SQLite: first insert=250, second insert=0, stored=250, idempotent=True
EDA: generated 4 figures; peak decade=1970s
{
  "collected_api_records": 250,
  "published_kafka_messages": 253,
  "consumed_kafka_messages": 253,
  "valid_records": 250,
  "invalid_records": 3,
  "records_after_cleaning": 250,
  "stored_sqlite_records": 250,
  "second_load_inserted": 0,
  "idempotency_passed": true,
  "figures_generated": 4,
  "verification_checks": {
    "published_equals_expected": true,
    "consumed_equals_published": true,
    "valid_plus_invalid_equals_consumed": true,
    "cleaned_equals_stored": true,
    "idempotency": true
  },
  "all_checks_passed": true
}
(venv) n_n@fedora:~/Projects/dcp-sis2/data-collection-sis2$
```

data > {} raw_data.jsonl


```
{"release_id": "49fd4b0c-47fa-5d4b-a86c-fea8ec78dee9", "artist_mbid": "83d91898-7763-47d7-b03b-b92132375c47", "artist_name": "Pink Floyd", "artist_credit_display":
{"release_id": "592dd977-b2c3-509a-9e1d-d84f251b38be", "artist_mbid": "83d91898-7763-47d7-b03b-b92132375c47", "artist_name": "Pink Floyd", "artist_credit_display":
{"release_id": "634f2ba8-95fa-5124-8edd-d240f6b4d1bd", "artist_mbid": "83d91898-7763-47d7-b03b-b92132375c47", "artist_name": "Pink Floyd", "artist_credit_display":
{"release_id": "7cf89da2-2b88-5e65-a117-cca2a135e43b", "artist_mbid": "83d91898-7763-47d7-b03b-b92132375c47", "artist_name": "Pink Floyd", "artist_credit_display":
{"release_id": "ac224e4e-5066-571c-b776-5fce4e734f87", "artist_mbid": "83d91898-7763-47d7-b03b-b92132375c47", "artist_name": "Pink Floyd", "artist_credit_display":
{"release_id": "b1180e73-5488-5836-9d7a-cad3de16f9d5", "artist_mbid": "83d91898-7763-47d7-b03b-b92132375c47", "artist_name": "Pink Floyd", "artist_credit_display":
{"release_id": "b8e8e073-c055-5ecc-8677-6ba415c4f2c2", "artist_mbid": "83d91898-7763-47d7-b03b-b92132375c47", "artist_name": "Pink Floyd", "artist_credit_display":
{"release_id": "c3f13620-5e2f-5442-a39e-c3a51680c6fc", "artist_mbid": "83d91898-7763-47d7-b03b-b92132375c47", "artist_name": "Pink Floyd", "artist_credit_display":
{"release_id": "e33cfb5c-dff4-5b5c-9e17-6c205d1610bc", "artist_mbid": "83d91898-7763-47d7-b03b-b92132375c47", "artist_name": "Pink Floyd", "artist_credit_display":
{"release_id": "e718aba2-c89c-53c0-bb28-68ba3550369a", "artist_mbid": "83d91898-7763-47d7-b03b-b92132375c47", "artist_name": "Pink Floyd", "artist_credit_display":
{"release_id": "fc234178-a998-5b1c-984f-6f9474b2ddd3", "artist_mbid": "83d91898-7763-47d7-b03b-b92132375c47", "artist_name": "Pink Floyd", "artist_credit_display":
{"release_id": "52fb5e82-cbf0-52a5-8302-2eefe66b15f7", "artist_mbid": "83d91898-7763-47d7-b03b-b92132375c47", "artist_name": "Pink Floyd", "artist_credit_display":
{"release_id": "71e8e4cc-2b1c-5ce8-9c57-569a616dc1a2", "artist_mbid": "83d91898-7763-47d7-b03b-b92132375c47", "artist_name": "Pink Floyd", "artist_credit_display":
{"release_id": "c1834ff7-9406-5d7c-b2a8-b29ca02e8487", "artist_mbid": "83d91898-7763-47d7-b03b-b92132375c47", "artist_name": "Pink Floyd", "artist_credit_display":
{"release_id": "8457b172-51d7-593c-beab-e995b89d69cc", "artist_mbid": "83d91898-7763-47d7-b03b-b92132375c47", "artist_name": "Pink Floyd", "artist_credit_display":
{"release_id": "c60535e1-6914-59ee-9bde-e087d37da76b", "artist_mbid": "83d91898-7763-47d7-b03b-b92132375c47", "artist_name": "Pink Floyd", "artist_credit_display":
{"release_id": "7ef1ffab-9ba2-5235-8871-e2b25959d87a", "artist_mbid": "83d91898-7763-47d7-b03b-b92132375c47", "artist_name": "Pink Floyd", "artist_credit_display":
{"release_id": "6f167370-3d5e-538e-af84-babfc05551c7", "artist_mbid": "83d91898-7763-47d7-b03b-b92132375c47", "artist_name": "Pink Floyd", "artist_credit_display":
{"release_id": "136712e6-e5f4-5c89-b144-6864f5e376b3", "artist_mbid": "83d91898-7763-47d7-b03b-b92132375c47", "artist_name": "Pink Floyd", "artist_credit_display":
{"release_id": "28dc87c0-f325-547c-9567-f52ace2fc5cd", "artist_mbid": "83d91898-7763-47d7-b03b-b92132375c47", "artist_name": "Pink Floyd", "artist_credit_display":
{"release_id": "2c6c988c-5a81-573d-8b05-69f1e96c5435", "artist_mbid": "83d91898-7763-47d7-b03b-b92132375c47", "artist_name": "Pink Floyd", "artist_credit_display":
{"release_id": "63b6d6c4-1810-5ba4-bbe0-2654cdb0186b", "artist_mbid": "83d91898-7763-47d7-b03b-b92132375c47", "artist_name": "Pink Floyd", "artist_credit_display":
{"release_id": "85f8ec6e-4c61-51e6-8350-52299fe4327f", "artist_mbid": "83d91898-7763-47d7-b03b-b92132375c47", "artist_name": "Pink Floyd", "artist_credit_display":
{"release_id": "89d1342f-35ae-5d5d-bc17-1f94cd321018", "artist_mbid": "83d91898-7763-47d7-b03b-b92132375c47", "artist_name": "Pink Floyd", "artist_credit_display":
{"release_id": "8a77cb2c-6ea8-5be0-8370-548c36374533", "artist_mbid": "83d91898-7763-47d7-b03b-b92132375c47", "artist_name": "Pink Floyd", "artist_credit_display":
{"release_id": "97f8763b-6551-5642-9ca6-e4eb87ca215f", "artist_mbid": "83d91898-7763-47d7-b03b-b92132375c47", "artist_name": "Pink Floyd", "artist_credit_display":
{"release_id": "a229470a-bd74-500a-be47-97afb7dd4ddd", "artist_mbid": "83d91898-7763-47d7-b03b-b92132375c47", "artist_name": "Pink Floyd", "artist_credit_display":
```

Task 3 - Kafka Streaming

Starting the broker and the topic

The broker is a real Apache Kafka 3.9.1 container defined in *docker-compose.yml*. After *docker compose up -d* I waited until the container *pink-floyd-kafka* became healthy, and then created and inspected the topic explicitly with *kafka-topics.sh*: one partition and a replication factor of 1, which is enough for a single local broker.

```
Run 'docker compose COMMAND --help' for more information on a command.
unknown docker command: "compose pd"
(venv) n_n@fedora:~/Projects/dcp-sis2/data-collection-sis2$ docker compose ps
NAME               IMAGE                COMMAND                  SERVICE   CREATED         STATUS                   PORTS
pink-floyd-kafka   apache/kafka:3.9.1   "/__cacert_entrypoin…"   kafka     3 minutes ago   Up 3 minutes (healthy)   0.0.0.0:9092→9092/tcp, [::]:9092→9092/tcp
(venv) n_n@fedora:~/Projects/dcp-sis2/data-collection-sis2$
```

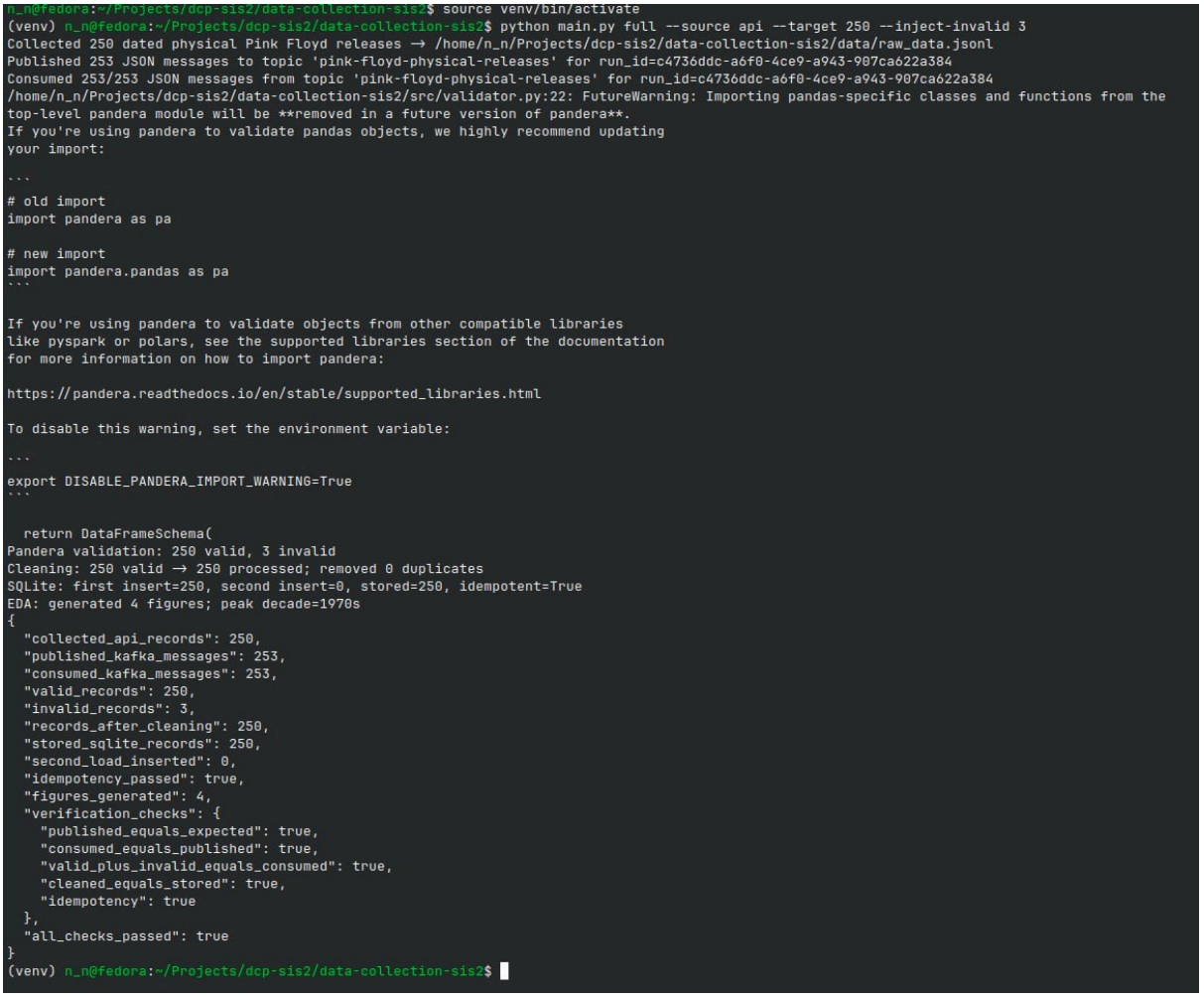
```
(venv) n_n@fedora:~/Projects/dcp-sis2/data-collection-sis2$ docker compose ps
NAME               IMAGE                COMMAND                  SERVICE   CREATED         STATUS                   PORTS
pink-floyd-kafka   apache/kafka:3.9.1   "/__cacert_entrypoin…"   kafka     3 minutes ago   Up 3 minutes (healthy)   0.0.0.0:9092→9092/tcp, [::]:9092→9092/tcp
(venv) n_n@fedora:~/Projects/dcp-sis2/data-collection-sis2$ docker compose exec kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --create --if-not-exists \
  --topic pink-floyd-physical-releases \
  --partitions 1 \
  --replication-factor 1
(venv) n_n@fedora:~/Projects/dcp-sis2/data-collection-sis2$ ^[[200~docker compose exec kafka /opt/kafka/bin/kafka-topics.sh \
>   --bootstrap-server localhost:9092 \
>   --describe --topic pink-floyd-physical-releases
bash: docker: command not found...

(venv) n_n@fedora:~/Projects/dcp-sis2/data-collection-sis2$ ~
bash: /home/n_n: Is a directory
(venv) n_n@fedora:~/Projects/dcp-sis2/data-collection-sis2$ docker compose exec kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --describe --topic pink-floyd-physical-releases
Topic: pink-floyd-physical-releases     TopicId: 9-tkc4Y3TZiEPR82w0vD7g PartitionCount: 1       ReplicationFactor: 1    Configs:
        Topic: pink-floyd-physical-releases     Partition: 0    Leader: 1       Replicas: 1     Isr: 1  Elr:    LastKnownElr:
(venv) n_n@fedora:~/Projects/dcp-sis2/data-collection-sis2$
```

Separate producer and consumer

The producer and the consumer are two independent scripts, which I demonstrated in two terminals. Both use the same **run ID**, so the consumer ignores stale messages from other runs. The consumer is started **first** and waits for exactly **253 messages** (250 source records + 3 deliberately invalid test messages). It has two termination safeguards: it stops when *consumed == expected*, and a 45-second idle timeout raises an error instead of hanging forever.

The *main.py full* command does the same in one run: it collects 250 releases, starts the finite consumer first, publishes 253 messages, waits for exactly 253 matching messages and continues with the validation, the cleaning, the double loading and the analysis.



Task 4 - Validation with Pandera

Schema

The module *src/validator.py* defines **14 Pandera rules**, including:

- UUID format and uniqueness of *release_id*;
- exact artist name and a required non-empty title;
- allowed release statuses and allowed physical format categories;
- ranges of the number of media and tracks;
- format of the country code;
- date pattern and date precision, and a valid release year range;
- data type coercion.

```python
def build_schema() -> DataFrameSchema:
    """Pandera schema with type, required, range, category, date, UUID and uniqueness rules."""
    return DataFrameSchema(
        {
            "release_id": Column(str, checks=Check.str_matches(UUID_PATTERN), unique=True, nullable=False),
            "artist_mbid": Column(str, checks=Check.str_matches(UUID_PATTERN), nullable=False),
            "artist_name": Column(str, checks=Check.eq("Pink Floyd"), nullable=False),
            "title": Column(str, checks=Check.str_length(min_value=1, max_value=300), nullable=False),
            "status": Column(str, checks=Check.isin(ALLOWED_STATUSES), nullable=False),
            "format_detail": Column(str, checks=Check.str_length(min_value=1), nullable=False),
            "format_category": Column(str, checks=Check.isin(sorted(PHYSICAL_FORMATS)), nullable=False),
            "media_count": Column(int, checks=Check.in_range(1, 20), nullable=False, coerce=True),
            "track_count": Column(int, checks=Check.in_range(1, 500), nullable=False, coerce=True),
            "country_code": Column(str, checks=Check.str_matches(r"^[A-Z]{2}$"), nullable=True),
            "release_date": Column(str, checks=Check.str_matches(DATE_PATTERN), nullable=False),
            "release_year": Column(int, checks=Check.in_range(1960, CURRENT_YEAR), nullable=False, coerce=True),
            "date_precision": Column(str, checks=Check.isin(["year", "month", "day", "partial"]), nullable=False),
            "is_test_invalid": Column(bool, nullable=False, coerce=True),
        },
        strict=False,
        coerce=False,
        ordered=False,
        name="physical_pink_floyd_releases",
    )
```

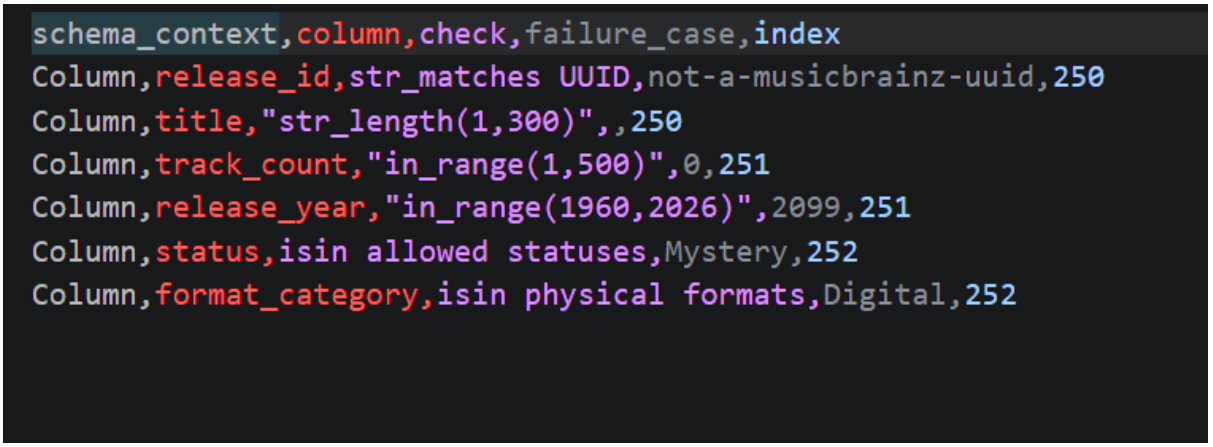
Three deliberately invalid messages

To prove that the validation really works, the producer injects three invalid test messages, each of them breaking different rules:

- invalid UUID and empty title;
- zero tracks and a future date;
- a disallowed status and a disallowed (digital) format.

Invalid records and all failure cases are written to *data/invalid_records.csv* and *data/validation_errors.csv*, and they never enter the processed table. As a result, 250 records are valid and 3 are invalid.





Task 5 - Cleaning and Transformation

The module *src/cleaner.py* prepares the valid records. The steps are:

- handling optional missing values, and normalizing whitespace and categories;
- converting data types and removing duplicates;
- NumPy calculations of age, decade, era and z-score;
- a multi-condition filter of recent multi-disc releases;
- a GroupBy aggregation;
- a pivot table of decade by format.

After the cleaning, **250 records** remain, so no valid record was lost. The result is saved as *data/processed_data.csv*.

```python
def clean_and_transform(
    input_path: Path,
    output_path: Path | None = None
) -> tuple[pd.DataFrame, dict]:

    output_path = output_path or DATA_DIR / "processed_data.csv"
    frame = pd.read_csv(input_path, dtype={"release_id": "string", "barcode": "string", "catalog_number": "string"})


    # for copmarison
    input_rows = len(frame)
    missing_before = frame.isna().sum().to_dict()


    # nullable strings are imputed using Unknown
    optional_text = ["country_code", "label", "catalog_number", "barcode", "primary_type"]
    for column in optional_text:
        frame[column] = frame[column].fillna("Unknown").astype(str).str.strip()


    # remove extra spaces
    for column in ["title", "release_group_title", "label", "format_detail"]:
        frame[column] = frame[column].fillna("").astype(str).map(lambda value: re.sub(r"\s+", " ", value).strip())

    # strip cols
    frame["artist_name"] = frame["artist_name"].str.strip()
    frame["status"] = frame["status"].str.strip().str.title()
    frame["format_category"] = frame["format_category"].str.strip()


    # Type conversion and dedup by UID
    frame["release_date_parsed"] = pd.to_datetime(frame["release_date"], errors="coerce")
    frame["release_year"] = pd.to_numeric(frame["release_year"], errors="raise").astype("int64")
    frame["track_count"] = pd.to_numeric(frame["track_count"], errors="raise").astype("int64")
    frame["media_count"] = pd.to_numeric(frame["media_count"], errors="raise").astype("int64")
    duplicate_rows = int(frame.duplicated(subset=["release_id"]).sum())
    frame = frame.drop_duplicates(subset=["release_id"], keep="first").copy()


    # nump transformations used by analysis
    frame["release_age_years"] = np.maximum(0, CURRENT_YEAR - frame["release_year"].to_numpy())
    frame["release_decade"] = (frame["release_year"] // 10 * 10).astype("int64")
    frame["era"] = np.select(
        [frame["release_year"] < 1980, frame["release_year"] < 2000],
        ["Classic 1960s-1970s", "Catalog 1980s-1990s"],
        default="Modern 2000s+",
    )
    frame["is_multi_disc"] = (frame["media_count"] > 1).astype("int64")
    track_std = float(frame["track_count"].std(ddof=0))
    frame["track_count_zscore"] = (
        (frame["track_count"] - frame["track_count"].mean()) / track_std if track_std else 0.0
    )
```

```python
# multi-condition filter, group aggregation and pivot-table reshaping
filtered = frame.loc[
    (frame["release_year"] >= 2000)
    & (frame["media_count"] > 1)
    & (frame["track_count"] >= 10)
].copy() # special releases with more than 1 disc after 2000 (disbandment)

# save to csv
filtered.to_csv(DATA_DIR / "filtered_recent_multidisc.csv", index=False)


grouped = (
    frame.groupby("format_category", as_index=False)
    .agg(release_count=("release_id", "count"), average_tracks=("track_count", "mean"), median_age=("release_age_years", "median"))
    .sort_values("release_count", ascending=False)
)
# svae to csv
grouped.to_csv(DATA_DIR / "groupby_format_summary.csv", index=False)


pivot = pd.pivot_table(
    frame,
    index="release_decade",
    columns="format_category",
    values="release_id",
    aggfunc="count",
    fill_value=0,
)
# save to csv
pivot.to_csv(DATA_DIR / "pivot_decade_format.csv")


frame["release_date_parsed"] = frame["release_date_parsed"].dt.strftime("%Y-%m-%d")
frame.to_csv(output_path, index=False)

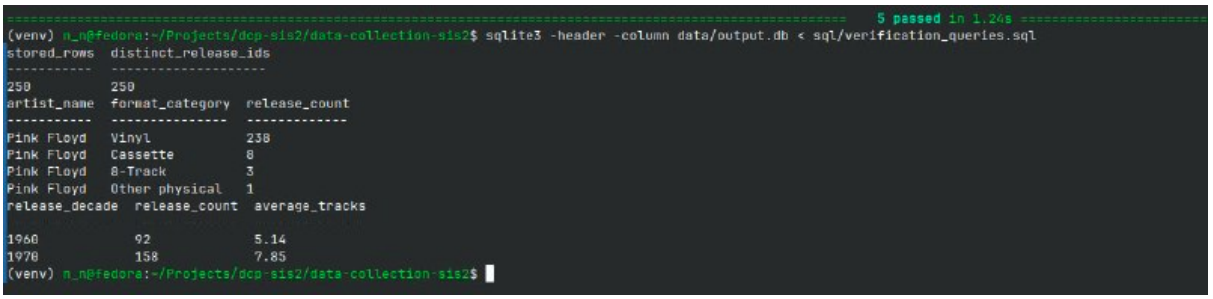

summary = {
    "input_valid_records": int(input_rows),
    "duplicates_removed": duplicate_rows,
    "records_after_cleaning": int(len(frame)),
    "missing_optional_values_before_fill": {key: int(value) for key, value in missing_before.items() if value},
    "multi_condition_filter_records": int(len(filtered)),
    "groupby_rows": int(len(grouped)),
    "pivot_shape": [int(pivot.shape[0]), int(pivot.shape[1])],
    "numpy_operations": ["release age", "decade", "era selection", "multi-disc flag", "track-count z-score"],
}
(DATA_DIR / "cleaning_summary.json").write_text(json.dumps(summary, indent=2), encoding="utf-8")
print(f"Cleaning: {input_rows} valid -> {len(frame)} processed; removed {duplicate_rows} duplicates")

return frame, summary
```

Task 6 - SQLite Database and SQL Verification

The loader creates the database *data/output.db* and the table *releases*. **release_id is the primary key**, and the insertion uses *ON CONFLICT(release_id) DO NOTHING*, which makes the load **idempotent**. To prove this, *python main.py process* intentionally loads the same processed table twice: the first load inserts 250 rows and the second inserts **0**, so the table stays at 250 rows.

The SQL checks are in *sql/verification_queries.sql* and are run with the SQLite CLI (*sqlite3 -header -column data/output.db < sql/verification_queries.sql*). They count the rows and the distinct release IDs (expected 250 and 250) and look for duplicated IDs (expected: an empty list).

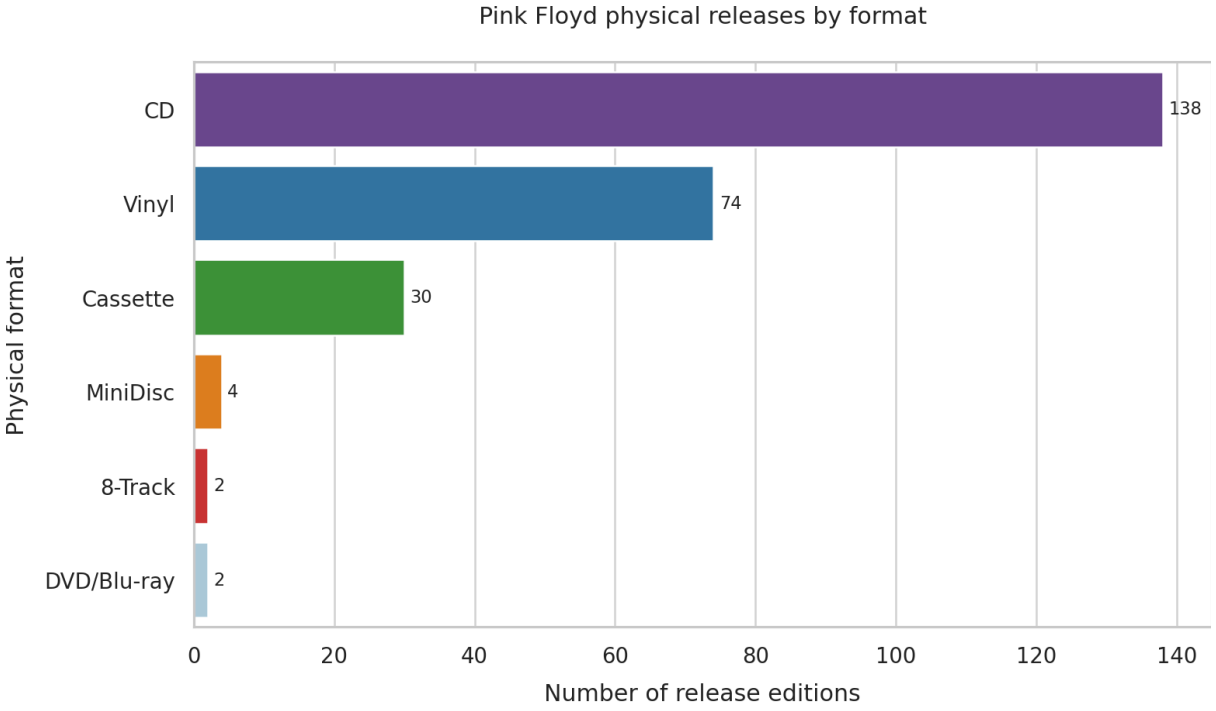


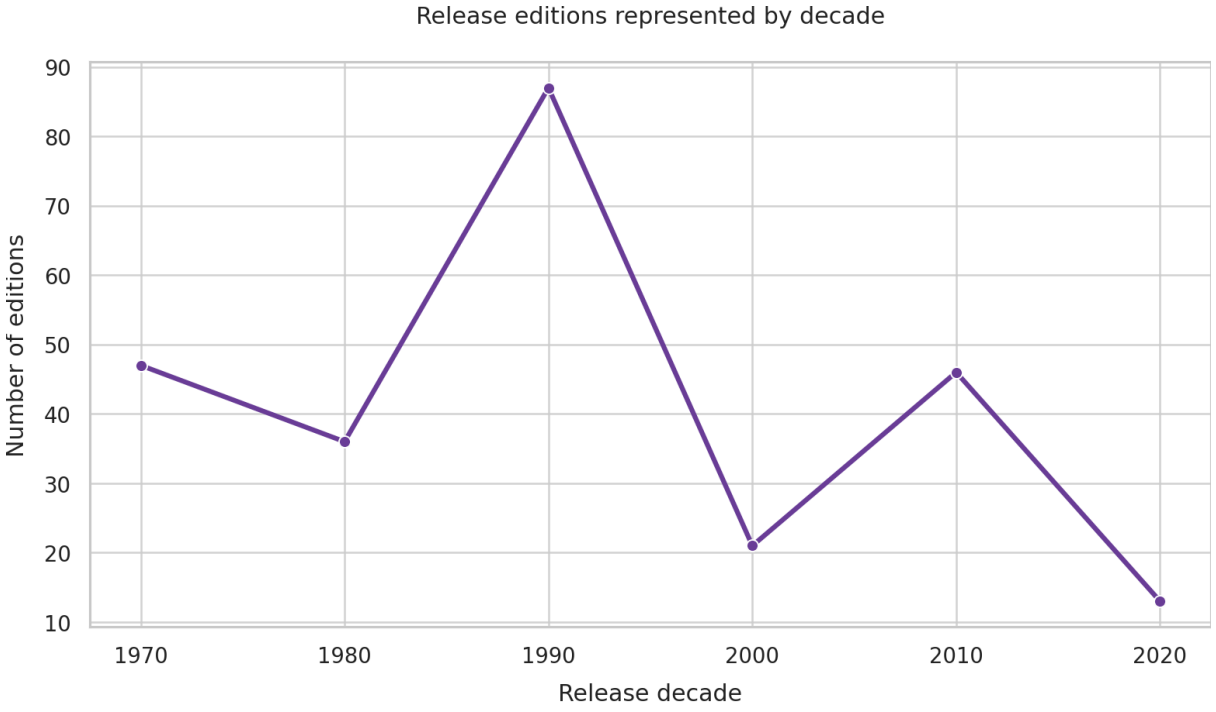
Task 7 - Exploratory Data Analysis

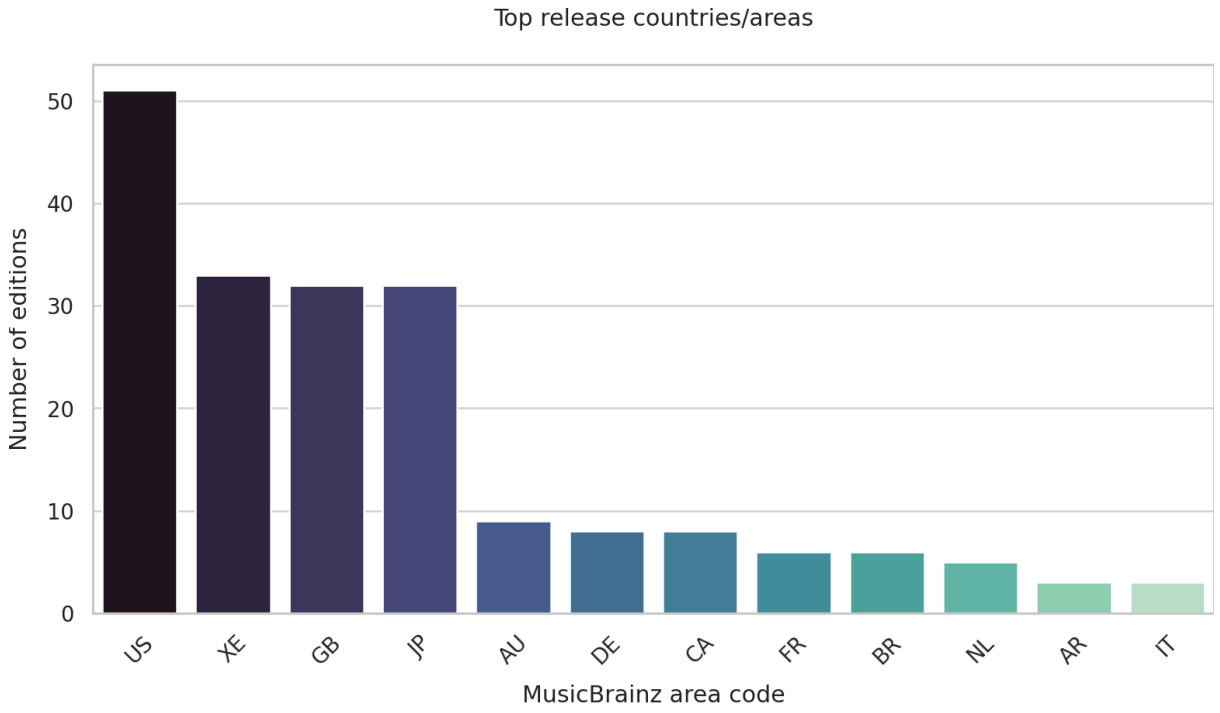
The analysis module (*python -m src.analysis --input data/processed_data.csv*) creates four figures in the *figures/* folder, plus *data/eda_statistics.json* and *data/eda_findings.md*:

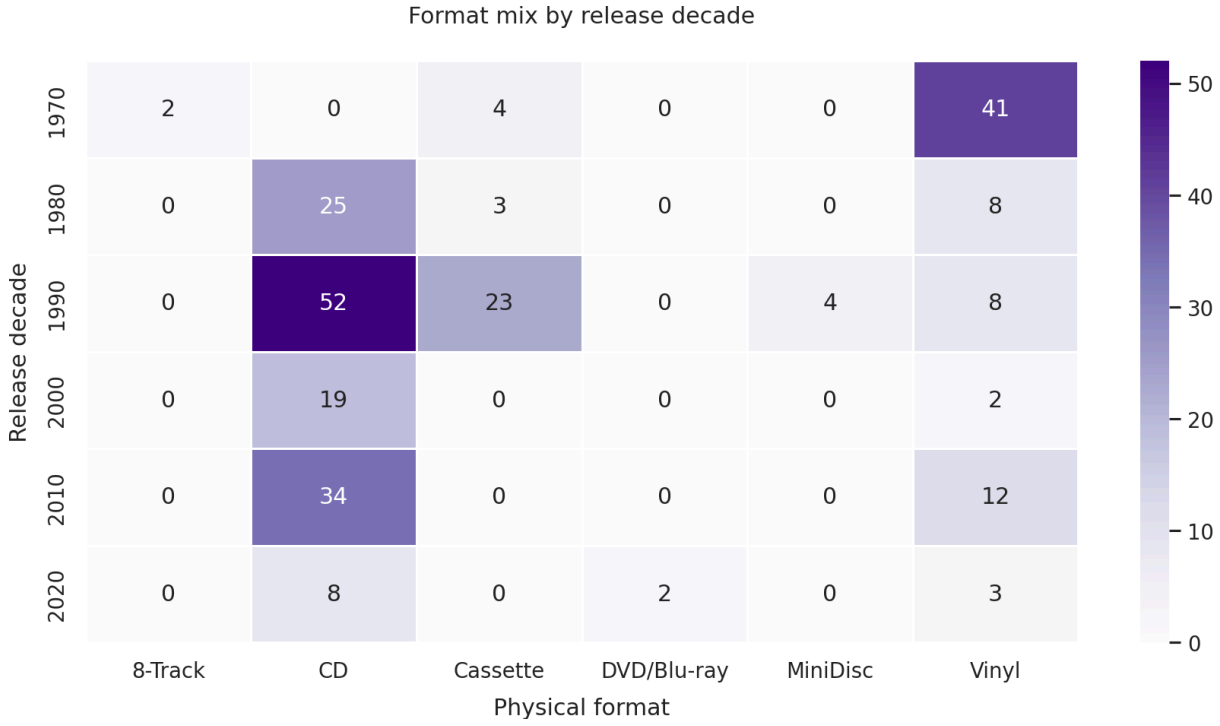
- *01_format_distribution.png* - number of releases by format;
- *02_releases_by_decade.png* - number of releases by decade;
- *03_top_countries.png* - the countries or areas with the most releases;
- *04_decade_format_heatmap.png* - a heatmap of decade by format.

The packaged 250-row run spans the years **1973-2025**. **CD** is the most frequent format (138 releases), the **1990s** are the peak decade (87 releases), and **36** country or area codes are represented.


Pink Floyd physical releases by format
Physical format
CD
Vinyl
Cassette
MiniDisc
8-Track
DVD/Blu-ray
138
74
30
4
2
2
0
20
40
60
80
100
120
140
Number of release editions


Release editions represented by decade
Number of editions
90
80
70
60
50
40
30
20
10
1970
1980
1990
2000
2010
2020
Release decade


Top release countries/areas
Number of editions
0
10
20
30
40
50
US
XE
GB
JP
AU
DE
CA
FR
BR
NL
AR
IT
MusicBrainz area code


Format mix by release decade
Release decade
1970
1980
1990
2000
2010
2020
2 0 4 0 0 41
0 25 3 0 0 8
0 52 23 0 4 8
0 19 0 0 0 2
0 34 0 0 0 12
0 8 0 2 0 3
8-Track
CD
Cassette
DVD/Blu-ray
MiniDisc
Vinyl
Physical format
0
10
20
30
40
50

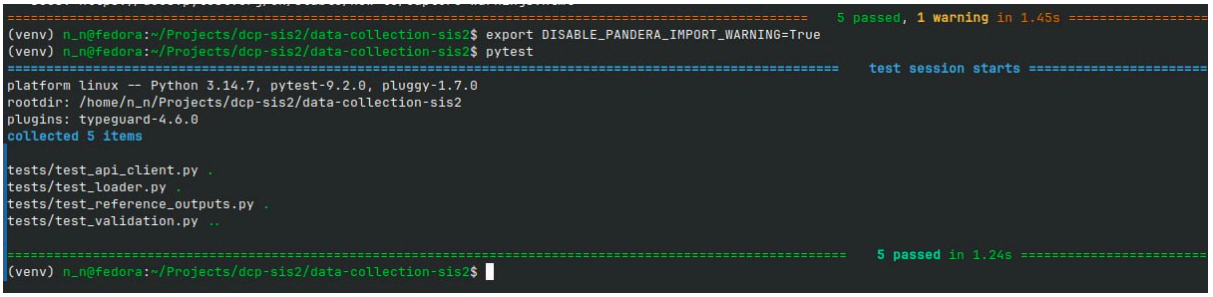
Task 8 - End-to-End Verification and Tests

The success of the whole pipeline is verified in *data/pipeline_summary.json*, where every check must be *true*. The expected counts of the snapshot run are shown in the table below. The three-message difference between the collected records and the Kafka messages is intentional: these are the invalid test messages.

Stage	Count
Source records collected	250
Kafka messages published	253
Kafka messages consumed	253
Valid records	250
Invalid test records	3
Records after cleaning	250
SQLite rows	250
Rows inserted by second load	0
Figures generated	4

```
{
  "collected_api_records": 250,
  "published_kafka_messages": 253,
  "consumed_kafka_messages": 253,
  "valid_records": 250,
  "invalid_records": 3,
  "records_after_cleaning": 250,
  "stored_sqlite_records": 250,
  "second_load_inserted": 0,
  "idempotency_passed": true,
  "figures_generated": 4,
  "verification_checks": {
    "published_equals_expected": true,
    "consumed_equals_published": true,
    "valid_plus_invalid_equals_consumed": true,
    "cleaned_equals_stored": true,
    "idempotency": true
  },
  "all_checks_passed": true,
  "reference_only": true,
  "kafka_broker_verified_in_build_environment": false,
  "pandera_verified_in_build_environment": false,
  "executed_in_build_environment": [
    "pandas/NumPy cleaning",
    "SQLite double load and SQL verification",
    "EDA and figure generation",
    "PDF generation and rendering"
  ]
}
```


The tests are run with *pytest*. They cover the format classification, the deliberate invalid records, the cleaning outputs, the structure of the schema and the idempotency of the SQLite loading.



Task 9 - Conclusions

Results

The pipeline works from the beginning to the end on a real Kafka broker: 250 dated physical releases were collected, 253 messages were published and consumed, the 3 invalid test messages were detected and quarantined, 250 valid records were cleaned and loaded into SQLite, a repeated load inserted 0 rows, and 4 figures were generated. The analysis shows that the catalog is dominated by CDs (138 of 250) and by the releases of the 1990s (87 of 250).

Limitations

- The dependencies in *requirements.txt* are **not pinned**, so a future version change of a library could break the code.
- The reported numbers come from the **250-record snapshot**; a live run can give slightly different results if the MusicBrainz catalog changes.
- The Kafka setup is a **single local broker** with one partition, so it demonstrates the data flow but not the scalability or fault tolerance of Kafka.
- The 3 invalid messages are **artificial test cases**, so the validation is proven on known errors, not on all the real errors that could appear in the API.
- MusicBrainz limits the request rate and can answer with 429 or 503 errors, so the live collection is slow and may need to be repeated.

Conclusion

The main lesson is that every stage of a pipeline must be checked independently: exact message counts between the producer and the consumer, a quarantine for invalid records, a primary key with an idempotent insert, and a machine-readable summary of all checks. This makes the whole pipeline reproducible and easy to defend.